

SC4063: Network Security

Week 1 : Intro to Network Security

Chua Zong Fu



© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

Welcome to SC4063 Network Security. I want to set the stage very clearly for what we are about to undertake.

This week, I will briefly walk you through what to expect for the entire module, which is about modern network security. The keyword is Modern.

For those that haven't had any hands on experience in cyber, don't worry. I will share real world experience on what works and what doesn't work, so you don't have to go through the same pain.



Why Network Security Had to Change

Modern Network Security: The Death of the Perimeter

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

Mental models we built in the 1990s and 2000s, the models of the 'trusted LAN,' the 'perimeter firewall,' and 'security by obscurity,' are not just outdated; they are dangerous. In this module, we are going to systematically dismantle those assumptions. We are moving from a world where security was defined by *location*, IP addresses and subnets, to a world where security is defined by *identity*, API keys, OAuth tokens, and ephemeral credentials.

This course serves as a bridge. We will take the foundational principles from the Stallings text, concepts like the CIA triad, Attack Trees, and Saltzer and Schroeder's Design Principles, and we are going to rigorously apply them to the chaotic reality of Hybrid Cloud, Kubernetes, and Zero Trust architectures. We aren't just here to learn how to block a port; we are here to learn how to design architectures that survive when the port is inevitably bypassed. Let's begin by looking at why our old maps no longer match the territory."



The "Castle & Moat" Fallacy

- Legacy Model: Security defined by IP location. Internal = Trusted; External = Untrusted
- Modern Reality: Users are mobile (e.g logging in from Starbucks), Data is SaaS (Snowflake), Infrastructure is IaaS (AWS)
- The Failure: Hard exterior, soft interior (The "M&M" model)
- Case Study: [Capital One Breach](#) (2019)
 - Mechanism: Attackers exploited a WAF misconfiguration to query the internal AWS Metadata Service (IMDSv1) [T1552.005]
 - The Assumption: The WAF was "trusted" because it was inside the perimeter
 - Critical Failure: The network allowed the WAF to talk to the Metadata service without authentication.
 - Unsecured Credentials: Cloud Instance Metadata API.

- What assumption failed here? That "Inside the Firewall" means "Trusted."

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

For thirty years, we taught network security using the 'Castle and Moat' analogy. The idea was simple: you have a hard outer shell, your perimeter firewall and a soft, trusted center. If a packet originated from inside the castle, we trusted it. If it came from outside, we inspected it. That model is dead. It didn't die because the firewalls stopped working; it died because the 'castle' evaporated. Today, your user is sitting in a Starbucks on public Wi-Fi. Your data is sitting in an AWS S3 bucket. There is no perimeter to defend anymore.

Consider the Capital One breach of 2019. This was a watershed moment. Capital One had a 'perimeter', a Web Application Firewall. But the attacker, Paige Thompson, found a Server-Side Request Forgery vulnerability. She tricked the server into talking to the AWS Metadata Service, the internal identity provider of the cloud instance. Because the network assumed that 'inside means trusted,' the WAF allowed the server to request its own temporary administrative credentials. The breach didn't happen because the firewall failed to block traffic; it happened because the architecture assumed that location equaled trust. The queried the Cloud Instance Metadata API, a vector that simply doesn't exist in traditional on-prem networks."



Why Layer 3, 4 Thinking Failed Us

- Limitations of the 5-Tuple: Src IP, Dst IP, Src Port, Dst Port, Protocol.
- Blind Spot: L3/L4 controls see connection, not intent or payload.
- The "Allow" Trap: Rules like Allow UDP 53 (DNS) or Allow TCP 443 (HTTPS) are permissive tunnels for malware.
- Case Study: [OilRig APT \(Helix Kitten\)](#)
 - Mechanism: DNS Tunneling. Encoding stolen data into the subdomains of DNS queries (e.g [stolen_data].attacker.com). [T1041]
 - The Failure: Firewalls allowed the traffic because "DNS is essential infrastructure" [T1071.004]

- What assumption failed here? That essential infrastructure ports are safe.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

In traditional networking, we live and die by the '5-tuple': Source IP, Destination IP, Source Port, Destination Port, and Protocol. We write Access Control Lists that say, 'Allow traffic on Port 443' because Port 443 is HTTPS, and the web needs HTTPS to function. The problem is that Layer 3 and 4 controls are context-blind. They see the *connection*, but they do not see the *intent*. To a Layer 4 firewall, a banking transaction and a Command and Control beacon look exactly the same, they are both encrypted streams of TCP packets.

Look at the OilRig Advanced Persistent Threat group. They utilized DNS Tunneling. Every firewall allows DNS traffic on UDP Port 53 because without DNS, the internet breaks. OilRig exploited this by encoding stolen data into the subdomains of DNS queries. To the Layer 4 firewall, it looked like legitimate address lookups. To the Layer 7 analyst, it was massive data exfiltration. This illustrates why looking at ports is no longer sufficient. If your security strategy relies solely on 'blocking ports,' you are leaving the door wide open for any attacker smart enough to use a standard protocol."



The Shift in Connectivity: Identity and API Calls Replaced Ports

- Old World: Open Port 22 (SSH) or 3389 (RDP) to manage servers.
- New World: Trigger `ssm:StartSession` or `kubectl exec`. No open inbound ports.
- The New Packet: The API Request (JSON).
- The New Router: The IAM Policy Engine.
- Case Study: [GitHub Security: Stolen OAuth User Tokens](#) (2022)
 - Mechanism: Attackers stole OAuth user tokens issued to third-party apps (Heroku, Travis CI). [T1550.001]
 - Impact: Used tokens to authenticate to GitHub API and clone private repos.
 - The Failure: Network controls are irrelevant when the attacker has the Identity key.

- What assumption failed here? That network controls stop application access.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

If ports are dead, what replaced them? The answer is Identity and APIs. In a modern cloud environment, you rarely SSH into a server. Instead, you make an API call. You send a JSON payload to an endpoint that says, 'Create a Load Balancer' or 'Read this Database.' The network security device that governs this isn't a router; it's an IAM Policy Engine. The packet isn't a TCP segment; it's a RESTful API request. This shift changes everything.

In the old world, if I wanted to stop you from accessing the database, I blocked port 3306. In the new world, I have to deny the `dynamodb:GetItem` action in your IAM policy. The GitHub OAuth attack in 2022 serves as a prime example of MITRE T1550.001, "Use Alternate Authentication Material". The attackers didn't scan for open ports on GitHub's servers. They stole OAuth tokens, essentially the digital keys that authorize API access. With those tokens, they could clone private repositories. The network was irrelevant. The 'port' was open to the world because it had to be. The security failure was in the Identity layer. Identity is the new firewall.



Data Flow Diagrams (DFD) Beat Network Diagrams

- Network Diagram: Shows cables, switches, and IP addresses. (Physical).
- DFD: Shows logic, data inputs, and trust crossings. (Logical).
- Why DFDs Win: Attackers exploit logic, not just topology.
- Case Study: [TJX Companies \(2007\)](#)
 - The Flow: Credit card data moved from stores to HQ via unencrypted Wi-Fi (WEP). [T1011]
 - The Gap: Network diagrams showed "Connectivity"; DFDs would have shown "Unencrypted Sensitive Data crossing Air Gap."

- Follow the data, not the wire.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

Veterans, stop showing me Visio diagrams of routers. I don't care about the router. I care about the Data. A Network Diagram tells me you have a switch. A Data Flow Diagram (DFD) tells me you are sending unencrypted credit card numbers over Wi-Fi. This is how TJX (TJ Maxx) got hacked. Their network map looked fine, connectivity was established. But they didn't map the *data flow*. If they had, they would have seen that sensitive PCI data was flying through the air protected only by WEP encryption, which is trivially breakable. DFDs expose the logical vulnerabilities that network maps hide.



From Implicit Trust (VPN) to Continuous Verification

- Definition: "If you have the credential to connect, you are trusted."
 - Flaw: One compromised endpoint = Total Network Access.
- Continuous Verification (The Zero Trust Model):
 - Definition: "Prove your Identity + Device Health + Context for every request."
 - Shift: Authentication is not a gate; it is a turnstile for every packet.
- Case Study: [US Energy Dept: Colonial Pipeline Analysis](#) (2021)
 - Vector: A legacy VPN profile with no MFA. [T1078] Valid Accounts (VPN).
 - Impact: Attackers gained entry via a single password and moved laterally to deploy ransomware.
 - Root Cause: The network implicitly trusted the VPN connection.

We used to operate on 'Implicit Trust.' This is the VPN model. You type your password, the VPN connects, and suddenly you have a virtual cable plugged into the data center. You are 'on the network,' and because you are on the network, you can ping the domain controller, access the file share, and scan the intranet. This model is catastrophic in the modern threat landscape. Once an attacker compromises a single endpoint, say, a laptop via a phishing email, the VPN becomes their bridge into your soft, gooey center.

We are moving to 'Continuous Verification,' often called Zero Trust. In a Zero Trust architecture, we assume the network is already hostile. We don't care if you are the CEO or a contractor. Every single request, for every single file, must be authenticated and authorized individually. We check not just your password, but the health of your device, your location, and the sensitivity of the data. The Colonial Pipeline ransomware incident happened because of a legacy VPN profile that had implicit trust. It lacked Multi-Factor Authentication. Once the attackers had that one password, they were 'inside.' Zero Trust kills the concept of 'inside.'



Redefining Traffic Directions: What 'East-West' Really Means Today

- Legacy East-West: Server-to-Server within a Data Center LAN.
- Modern East-West: SaaS-to-SaaS, Cloud-to-Cloud, Vendor-to-Customer.
- Risk: Integrations allow lateral movement between organizations.
- Case Study: [LAPSUS\\$ & Okta \(2022\)](#)
 - The Path: Compromised Okta Support Engineer (Sitel) [T1199] -> Okta Internal Tools -> Reset Customer Passwords [T1098]
 - The Pivot: Attackers moved "East-West" from the Vendor (Okta) to the Customer tenants.
 - Blind Spot: Customers trusted the "Okta Support" traffic as legitimate SaaS operations.

- What assumption failed here? That vendors are isolated from customers.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

In the data center era, 'East-West' traffic meant traffic moving between servers in the same rack. We largely ignored it because we trusted the physical security of the data center. Today, 'East-West' means something very different and much more dangerous. 'East-West' is now the traffic moving between your SaaS providers. It is your Okta tenant talking to your Slack workspace. It is your GitHub Actions runner deploying code to your AWS Lambda functions. This traffic often traverses the public internet, but it bypasses your corporate firewall entirely because it happens via API integrations.

The LAPSUS\$ group masterfully exploited this, leveraging MITRE T1199, Trusted Relationships. They didn't hack into companies directly; they hacked the identity providers and the support portals. By compromising a support engineer at Okta, they could pivot 'East-West' into the tenants of Okta's customers. They rode the trusted API connections that businesses rely on. The assumption that failed here is that your vendors are isolated from your core environment. They are not. In a hyper-connected SaaS ecosystem, your vendor's risk is your risk.



The Shared Responsibility Gap: Cloud Broke Perimeter Assumptions

- Premise: You secure the Data; Provider secures the Infrastructure.
- The Trap: Default settings often favor usability over security (e.g "Authenticated Users" = Everyone with an AWS account).
- Case Study: [Pegasus Airlines](#) (2022)
 - Incident: Misconfigured AWS S3 Bucket.
 - Exposure: 23 Million files (Flight charts, Crew PII, Source Code) exposed to the public internet.
 - Cause: "Public" access was enabled on a resource assumed to be internal [T1530]

- What assumption failed here? That creating a resource makes it private by default.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

When we owned the data center, we owned the hardware. We could physically unplug a server if we had to. In the cloud, we rely on the 'Shared Responsibility Model.' The cloud provider secures the concrete and the hypervisor; you secure the data and the configuration. The danger lies in the assumption that creating a resource in the cloud makes it private by default. In many cases, historically, it did not. Or, a simple configuration error, a single checkbox, could turn a private database into a public billboard.

Pegasus Airlines learned this in 2022. They had an AWS S3 bucket configured without proper access controls. It wasn't a sophisticated hack. It was a configuration drift. 23 million files, including flight charts and crew PII, were exposed to the entire internet. The 'perimeter' in the cloud is defined by policy, not by cables. If your policy is weak, or if your Terraform script has a typo, your perimeter vanishes instantly. This reinforces the need for 'Policy as Code' and automated compliance scanning.

In the real world, you cannot click through the AWS console to secure buckets. You must define infrastructure in code using Terraform or CloudFormation. We need to treat infrastructure with software engineering discipline: version control, code review, and automated testing. We know that we can use automated benchmarks like CIS Benchmarks, to continuously scan our environment. If a bucket becomes public, the automation alerts us or auto-remediates it immediately.



The Visibility Gap: Hybrid Creates Asymmetric Visibility

- On-Prem: Packets, Span Ports, Taps (Network Centric).
- Cloud: CloudTrail, VPC Flow Logs, API Logs (Identity Centric).
- The Seam: The connection between them (Direct Connect/VPN) is often unmonitored.
- Case Study: [Target Breach](#) (2013)
 - The Pivot: Attackers compromised a third-party HVAC vendor (Fazio Mechanical) [T1199]
 - The Path: Vendor Portal (External) -> Corporate Network (Internal) -> POS Network (Restricted) [T1021]
 - Failure: Lack of segmentation between the "Vendor" zone and the "Payment" zone.

- What assumption failed here? That 3rd party connections are lower risk.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

Most of you aren't running pure cloud environments. You are running Hybrid, a messy mix of on-premise legacy hardware and shiny new cloud infrastructure, connected by Direct Connect or VPN tunnels. This creates a problem of 'Asymmetric Visibility.' Your on-premise SOC sees packets via span ports and taps. Your cloud engineers see CloudTrail logs and VPC Flow Logs. But very often, nobody is watching the bridge between them. Attackers love this seam. They love to land in the cloud and pivot on-prem, or vice versa.

The classic Target breach illustrates this pivot, utilizing MITRE T1199, Trusted Relationship. The attackers didn't smash the front door of the payment network. They compromised a third-party HVAC vendor, an external, 'cloud-like' entity relative to Target. From that vendor's portal, they pivoted into the corporate network, and from there, moved laterally to the Point of Sale systems. The failure was assuming that the connection to the vendor was low-risk. In a hybrid world, every connection to a third party is a potential main entrance for an adversary.



The Credential Problem: Static Credentials in a Dynamic World

- Legacy: Service Accounts with non-expiring passwords hardcoded in configs.
- Modern Risk: Secrets sprawl in CI/CD pipelines, GitHub repos, and developer laptops.
- Case Study: [CircleCI Breach](#) (2023)
 - Vector: Malware on a developer's laptop stole a session cookie.
 - Impact: Attackers accessed the CircleCI database and stole environment variables (AWS Keys, API Tokens) for customers [T1539]
 - Lesson: A developer laptop is a Tier-0 Asset.

- What assumption failed here? That developer endpoints are not production entry points.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

In the old days, we generated a service account password, set it to never expire, and hardcoded it into a config file. That server sat in a closet for five years. Today, in a dynamic CI/CD environment, servers are born and die in minutes. But if we use static credentials in this dynamic world, we are planting time bombs. Developers often commit these static keys to code repositories. It's the path of least resistance. Look at the CircleCI breach in 2023. Malware on an engineer's laptop stole their session token. Because the session token (a static credential for that session) was valid, the attacker could impersonate the engineer and access the CircleCI databases. From there, they stole the environment variables-the API keys and secrets-of CircleCI's customers. The assumption that failed here is that developer endpoints are not production entry points. They absolutely are. If a developer has a commit key on their laptop, that laptop is a Tier 0 asset.



The Confidentiality Paradox: Encryption is Not a Silver Bullet

- Benefit: Encryption protects data from eavesdropping.
- Risk: Encryption protects malware from inspection (DLP/IDS evasion).
- Stat: ~90% of malware Command & Control (C2) is encrypted (SSL/TLS).
- Case Study: [Cobalt Strike C2](#)
 - Tactic: "Malleable C2 Profiles."
 - Technique: Mimicking legitimate traffic (e.g Amazon browsing, Google Search) inside an HTTPS tunnel [T1071.001]
 - Defense Requirement: TLS Inspection (Decryption) at the boundary.

- What assumption failed here? That encrypted traffic is benign.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

We tell our students: 'Encrypt everything.' And yes, confidentiality is critical. But as defenders, encryption is a double-edged sword. It protects data from eavesdropping, but it also protects attackers from *us*. Approximately 90% of modern malware traffic is encrypted. When an attacker exfiltrates gigabytes of data from your network, they wrap it in TLS. To your Intrusion Detection System (IDS), it looks like harmless web traffic. The Cobalt Strike framework, a favorite tool of both red teams and ransomware gangs, uses 'Malleable C2 profiles.' This maps to MITRE T1001.003, Protocol Impersonation. These profiles allow the attacker to shape their traffic to look exactly like legitimate traffic, for example, mimicking a Google search or an Amazon shopping cart transaction, all wrapped in HTTPS. If you are not performing TLS Inspection, decrypting, inspecting, and re-encrypting traffic at the boundary, you are blind. The assumption that 'encrypted traffic is safe traffic' is one of the most dangerous fallacies in modern cyber defense.



The Trusted Vendor Fallacy

- Concept: Organizations implicitly trust updates from major vendors (Microsoft, SolarWinds, CrowdStrike).
- The Attack: Compromising the build pipeline, not the end user.
- Case Study: [SolarWinds Orion \(Sunburst\)](#) (2020)
 - Mechanism: Malicious code injected into the source before compilation.
 - Result: The malware was digitally signed by a valid SolarWinds certificate [T1195.002]
 - Impact: Firewalls and AV trusted the binary because the signature was valid.

- What assumption failed here? That signed software updates are trustworthy.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

In late 2020, the cybersecurity world was shaken by the SolarWinds Orion supply chain attack. This incident profoundly changed how we view trust. SolarWinds was a trusted vendor. The attackers, a Russian SVR unit, didn't attack the customers directly. They compromised the build pipeline of SolarWinds itself.

They injected malicious code, the 'Sunburst' backdoor, into the source code of the Orion software. SolarWinds then compiled that code, *digitally signed it* with their valid certificate, and pushed it out as a standard update. Thousands of customers downloaded the update. Their firewalls allowed it because it came from a trusted source. Their antivirus ignored it because it was digitally signed. The assumption that failed here was absolute. We assumed that 'Signed Software' equals 'Safe Software.' We assumed that if we trust the vendor, we can trust the binary. That assumption is gone. We now must verify the integrity of the software supply chain itself.



Edge Appliances as Black Boxes

- Problem: Security appliances (VPNs, File Transfer) sit on the public internet [T1190]
- Blindness: You cannot install EDR agents on proprietary appliance OSs.
- Case Study: [MOVEit Transfer](#) (2023)
 - Vulnerability: SQL Injection (SQLi) in the web interface.
 - Actor: CL0P Ransomware Gang.
 - Impact: Webshells dropped on appliances allowed data theft without entering the LAN [T1505.003]

- What assumption failed here? That security appliances don't need defense-in-depth.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

For years, we bought 'Security Appliances', specialized boxes to handle file transfers, VPNs, and email. We put them on the edge of our network, exposed to the internet, thinking they were the shields. The MOVEit Transfer exploitation in 2023 taught us that these shields are often the weak points. MOVEit is a Managed File Transfer appliance used to move sensitive data. It sits on the public internet. It had a SQL Injection vulnerability, a basic web flaw mapped to MITRE T1190.

Because these appliances are 'black boxes' provided by vendors, we often can't install our own endpoint protection (EDR) agents on them. We can't see what's happening inside the OS of the appliance. So when the CL0P gang dropped a webshell onto these boxes (T1505.003), they had persistent access to the most sensitive data in the network, and the defenders were blind. The lesson here is that 'Edge Appliances' increase your attack surface. Every box you expose to the internet is a liability, not an asset.



Entropy in Security: Configuration Drift

- Concept: Systems degrade from "Secure" to "Insecure" over time due to human changes.
- Drivers: Debugging, Emergency fixes, Shadow IT, "Temporary" rules.
- Stallings Principle: Violation of "Economy of Mechanism" (Complexity hides flaws).
- Case Study: [Microsoft Azure "BlueBleed"](#) (2022)
 - Incident: Misconfigured Azure Blob Storage.
 - Impact: 2.4 TB of data exposed (Source code, Customer emails) [T1530]
 - Root Cause: Use of "Public" access setting on internal storage containers.

- What assumption failed here? That "set and forget" works in Cloud.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

Stallings discusses the design principle of 'Economy of Mechanism', keeping things simple. The cloud is the opposite of simple. It is infinitely complex and constantly changing. This leads to 'Configuration Drift.' You might deploy a secure environment on Day 1. On Day 2, a developer opens a port for debugging. On Day 3, a new feature requires a new IAM role. Over six months, your secure environment drifts into an insecure state.

This is exactly what happened in the 'BlueBleed' incident with Microsoft Azure. A misconfiguration in Azure Blob Storage meant that 2.4 Terabytes of data were publicly accessible. It wasn't a zero-day exploit. It wasn't a sophisticated hacker cracking a password. It was a configuration setting that drifted away from 'secure.' The assumption that 'Set and Forget' works in the cloud is fatal. We need continuous, automated monitoring, CSPM (Cloud Security Posture Management), to detect drift the second it happens.



The Soft Center: The Fallacy of the "Trusted LAN"

- Old View: Hard outer shell, soft gooey center.
- Legacy Protocols: SMB, WMI, and RDP allowed freely between workstations.
- Case Study: [NotPetya](#) (2017)
- Entry: Compromised Ukrainian accounting software (M.E.Doc).
- Propagation: Used EternalBlue (Exploit) [T1210] and Mimikatz (Credential Dumping) [T1003] to move laterally via SMB [T1021.002] .
- Speed: Flattened Maersk's global network (45,000 PCs) in ~7 minutes.

- What assumption failed here? That internal traffic is friendly.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

I want you to think about the 'Trusted LAN.' We grew up believing that the internal network was a safe haven. We put hard shells on the outside, but inside, we allowed SMB, RDP, and WMI to flow freely between workstations. The NotPetya attack in 2017 destroyed this concept forever. NotPetya was a wiper malware. It entered the network through a compromised piece of software. Once it was inside, it didn't just use exploits.

It utilized MITRE T1021.002, SMB Windows Admin Shares. It grabbed credentials from memory (T1003) and used legitimate administrator connections to jump from machine to machine. It flattened Maersk's global network in seven minutes. Seven minutes. That is the speed of a flat, trusted LAN. If you are not segmenting your internal network, if Workstation A can talk to Workstation B for no reason, you are building a superhighway for ransomware.



Infrastructure as Code (IaC) Risks

- Shift: Networks defined in Terraform, CloudFormation, YAML.
- Risk: A typo in code replicates a vulnerability across 1,000 servers instantly.
- Case Study: [Uber AWS Breach](#) (2022)
 - Discovery: Attackers found a PowerShell script on a network share [T1552.001]
 - Payload: Hardcoded Admin credentials for PAM (Thycotic).
 - Impact: Total compromise of AWS and GCP environments [T1078.004]

- What assumption failed here? That code repositories are just for logic, not secrets.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

In the modern era, we don't configure networks by typing commands into a CLI. We define networks in code, Terraform, CloudFormation, YAML. This is 'Infrastructure as Code' (IaC). It allows us to deploy massive environments in seconds. But it also allows us to replicate vulnerabilities at massive scale. If you make a mistake in a Terraform module that is used by 500 servers, you have just opened 500 security holes instantly. Consider the Uber breach of 2022. Attackers found a PowerShell script on a network share. Inside that script, which was a piece of infrastructure code, were hardcoded administrative credentials for their PAM (Privileged Access Management) system. This is MITRE T1552.001, Credentials in Files. The attackers used those credentials to gain 'God Mode' access to Uber's AWS and Google Cloud environments. The assumption here was that code repositories and scripts are just for logic. They are not. They are often graveyards of secrets.



Cloud Economics as an Indicator: Data Gravity & Egress

- Data Gravity: Data attracts apps, services... and attackers.
- Egress: Moving data out of the cloud costs money (and leaves logs).
- Case Study: [Code Spaces](#) (2014)
 - Incident: Attackers gained AWS Console Access.
 - Extortion: Demanded ransom. Code Spaces refused.
 - Impact: Attackers deleted S3 buckets (Production Data) and Snapshots (Backups) [T1485]
 - Outcome: Company ceased operations in 12 hours.

- What assumption failed here? That backups inside the same cloud account are safe

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

There is a concept in cloud computing called 'Data Gravity.' Data attracts applications, and it attracts services. It also attracts attackers. In the cloud, you often don't realize you've been breached until you get the bill or the system vanishes. In the Code Spaces breach of 2014, the company was destroyed not just because their data was deleted (MITRE T1485), but because the attackers gained control of the AWS console. They held the data hostage. When Code Spaces tried to fight back, the attackers deleted the backups. The backups were stored in the same cloud account as the production data. The assumption failed here on two fronts: first, that backups in the same failure domain are safe, and second, that we would notice the theft before the damage was done. We need to monitor 'Data Egress' and isolate backups in separate accounts (an 'Air Gapped' account) to survive this.



Identity as the Soft Target: The Human Perimeter

- Trend: Why exploit a kernel when you can ask the Help Desk for a password?
- Attack: Social Engineering / Vishing (Voice Phishing).
- Case Study: [MGM Resorts](#) (2023)
 - Actor: Scattered Spider (ALPHV affiliate).
 - Method: Found employee info on LinkedIn -> Called Help Desk -> Impersonated Employee -> Reset MFA [T1566.004]
 - Result: Full administrative access; \$100M+ loss.

- What assumption failed here? That technical controls stop social engineering.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

We spend millions on firewalls, EDR, and Zero Trust networking. But the easiest way into a network today is not to hack a computer; it is to hack a human. The 'Human Perimeter' is the softest part of your attack surface. The MGM Resorts breach in 2023 was a masterclass in this, utilizing MITRE T1566.004 (Spearphishing Voice). The group 'Scattered Spider' didn't write a zero-day exploit. They went on LinkedIn, found an employee, and called the IT Help Desk. They impersonated the employee, claimed they lost their phone, and asked for a Multi-Factor Authentication (MFA) reset. The Help Desk agent, trying to be helpful, reset the MFA. The attackers were in. They encrypted the casinos and cost MGM over \$100 million. We assume that 'Technical Controls' stop 'Social Engineering.' They do not. If your process allows a human to bypass a security control based on a phone call, your security control does not exist.



The Shadow API Problem: API, The Unmonitored Door

- Context: API traffic > HTML traffic.
- Vulnerability: BOLA (Broken Object Level Authorization). User A requests User B's data by changing an ID.
- Case Study: [Optus Breach](#) (2022)
 - Vector: A test API endpoint (api.optus.com.au) exposed to the internet [T1190]
 - Auth: None (Unauthenticated).
 - Method: ID Enumeration (Incrementing contact IDs).
 - Impact: 9.8 Million customer records stolen.

- What assumption failed here? That hidden URLs remain hidden.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

For the last decade, we have focused on securing the 'Front Door', the web application UI. We put WAFs in front of our websites. But modern apps are powered by APIs. These are the 'Back Doors,' and often, they are left unlocked. API traffic now exceeds HTML traffic on the internet. Yet, many organizations don't even have an inventory of their APIs. This leads to vulnerabilities like BOLA, Broken Object Level Authorization. The Optus breach in Australia is the textbook example. An API endpoint used for testing was left exposed to the internet. It didn't require authentication. Anyone who found the URL could request customer records. The attackers simply enumerated the IDs, 1, 2, 3, 4, and scraped 9.8 million records. The assumption that failed here is that 'Hidden URLs remain hidden.' 'Security by Obscurity' is not a defense when attackers run automated scanners 24/7.



Living off the Land (LOTL)

- Threat: Attackers using native tools to avoid detection.
- Tools: PowerShell, WMI, Bash, AWS CLI.
- Case Study: [Volt Typhoon \(2023\)](#)
 - Target: Critical Infrastructure (Guam).
 - Technique: Used built-in Windows tools (netsh, route) to proxy traffic. No malware files were dropped. [T1059]
 - Lesson: Whitelisting binaries is not enough; you must monitor behavior.

Threat modeling often focuses on 'Malware.' But sophisticated actors like Volt Typhoon don't use malware. They use *your* tools. They use PowerShell. They use netsh. This is 'Living off the Land.'

If your threat model assumes that Antivirus will catch the bad guy, you will fail. You need behavioral monitoring. Why is the print spooler executing a PowerShell script? Why is the web server running whoami? That is the anomaly."



Why Firewalls Became Insufficient Alone

- The Context Gap:
 - Firewall View: Sees TCP / 443 / Established.
 - WAF View: Sees HTTP / GET / ?query=\${jndi:ldap://...}.
 - The Failure: L3/L4 Firewalls cannot see inside the SSL/TLS tunnel or interpret application logic.
- Case Study: [Log4Shell Bypass \(2021\)](#)
 - Attack: String concatenation (`${lower:j}ndi...`) evaded signature-based WAFs. [T1027]
 - Lesson: Even L7 inspection is hard; L4 is useless against app exploits. What would have worked?
 - Input Validation: Sanitize data before it processes (Shift Left).
 - Egress Filtering: Why could the Log4j server talk to the attacker's LDAP server? Block unnecessary outbound traffic.
 - Runtime Application Self-Protection (RASP): Detect attacks executing inside the application memory.

- A firewall sees a handshake; an attacker sees a shell.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

For decades, the firewall was the king of security. But recall Slide 3, Layer 3 and 4 thinking failed us. Why? Because firewalls lack context. A firewall sees a TCP handshake on port 443. It sees encrypted data flowing. It marks the connection as 'Established' and walks away.

But inside that encrypted tunnel, the attacker is sending a Log4Shell exploit string. The firewall blindly passes it because the port is allowed. Even early Web Application Firewalls (WAFs) failed here. When Log4Shell hit, attackers used obfuscation, MITRE T1027. They split the malicious string into pieces, like `${lower:j}ndi`. Simple regex rules missed it. This proves that firewalls alone are insufficient. We need deep packet inspection, TLS decryption, and runtime application self-protection (RASP). We need tools that understand the *semantics* of the traffic, not just the *syntax* of the headers. Security must move up the stack to Layer 7 and beyond.

Defense-in-Depth applies to applications too. We need a WAF to catch the noise, but we fundamentally rely on secure coding and input validation. Furthermore, if the server had been properly segmented with strict Egress Filtering blocking the outbound LDAP connection, the exploit chain would have broken.



Serverless Security

- No Server ≠ No Risk:
 - Overview: Code executes within ephemeral functions (AWS Lambda), with the cloud provider managing OS patching.
 - Emerging Threats: Event Injection, excessive permissions, and Denial of Wallet attacks.
- Case Study: [Denial of Wallet](#)
 - Attack Vector: Initiating a recursive Lambda invocation loop.
 - Consequence: The function scales uncontrollably, rapidly exhausting the cloud budget. [T1059.004]
 - Mitigation: Implement concurrency limits and monitor billing alerts.

- In Serverless, you pay for the attack execution time.

© 2025 Nanyang Technological University, Singapore. All Rights Reserved.

Classification: Restricted

Serverless computing, like AWS Lambda, promises a world where we don't manage servers. And while we don't manage the OS, we still own the risk. Serverless introduces unique threats, like 'Denial of Wallet.'

In a traditional DDoS, your server crashes. In Serverless, your server scales. It scales to meet the demand of the attack. Amazon happily charges you for every millisecond of that attack. A recursive loop, where Function A calls Function B which calls Function A, can bankrupt a small startup in hours.

We also see 'Event Injection,' where attackers manipulate the JSON payloads that trigger these functions. Security here means defining strict concurrency limits (to stop the billing bleed) and validating every event trigger input.

This brings us to the end of Lecture 1. The goal of today's session was to give you a grounded view of how modern, real-world attacks succeed, not because attackers are magical, but because networks fail in predictable ways. From the earliest days of the Internet to today's cloud and hybrid environments, the network has always been the primary conduit for attacks, and that reality has not changed, even as the technologies around it have evolved.

In the next lecture, we will step back and revisit core networking fundamentals. This refresher is important, because you cannot defend what you do not properly

understand. After that, we will move into hands-on work, where you will get practical exposure to packet-level visibility using one of the most widely used and accessible tools in the field, tcpdump. This is where theory starts to connect with reality, so come prepared to get your hands dirty.